

IRIS_FLOWER_DATA_ANALYSIS

```
In [1]: import numpy as np
import pandas as pd
```

Importing pandas and Seaborn module

```
In [2]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

Importing Iris data set

```
In [3]: iris = pd.read_csv(r'D:\Samson - All Data\Naresh IT Institute\New folder\Iris.csv')
```

```
In [4]: iris
```

```
Out[4]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

Displaying data

In [5]: `iris.head()`

Out[5]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

In [6]: `iris.drop('Id',axis=1,inplace=True)`

In [7]: `iris.head()`

Out[7]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Checking if there are any missing values

In [10]: `iris.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   SepalLengthCm    150 non-null    float64
1   SepalWidthCm     150 non-null    float64
2   PetalLengthCm    150 non-null    float64
3   PetalWidthCm     150 non-null    float64
4   Species          150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

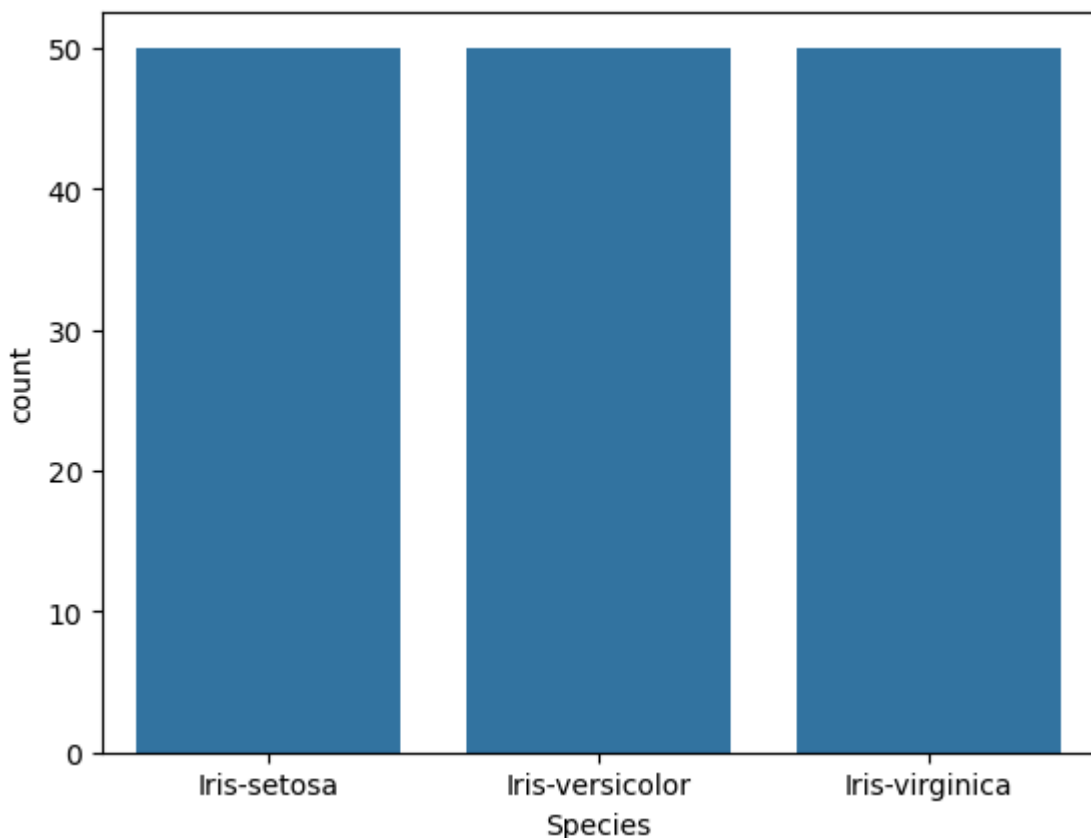
In [11]: `iris['Species'].value_counts()`

```
Out[11]: Species
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64
```

This data set has three varieties of Iris plant.

2.Bar Plot : Here the frequency of the observation is plotted. In this case we are plotting the frequency of the three species in the Iris Dataset

```
In [13]: sns.countplot(x='Species', data=iris)
plt.show()
```



We can see that there are 50 samples each of all the Iris Species in the data set.

4. Joint plot

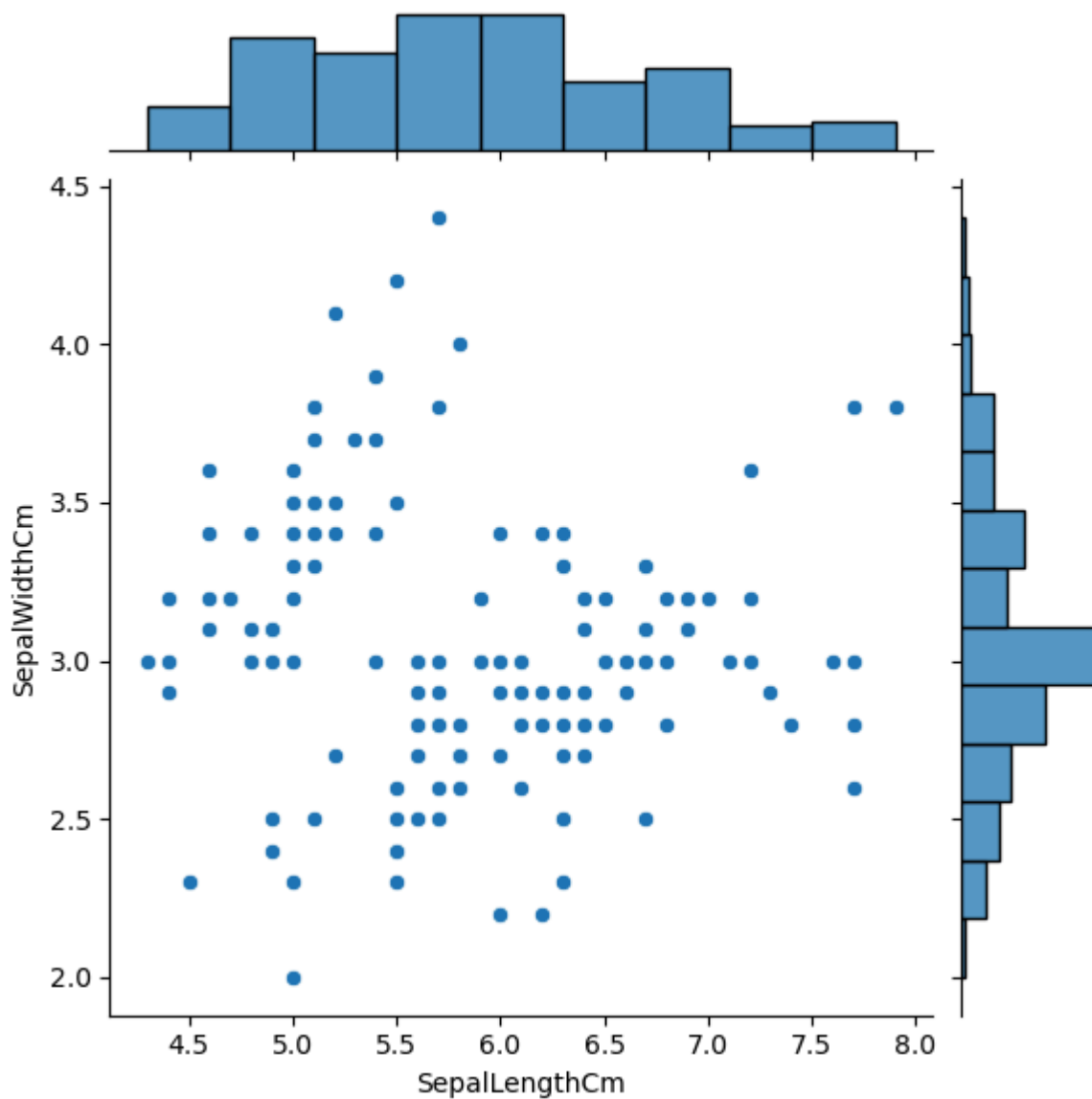
Jointplot is seaborn library specific and can be used to quickly visualize and analyze the relationship between two variables and describe their individual distributions on the same plot.

```
In [15]: iris.head()
```

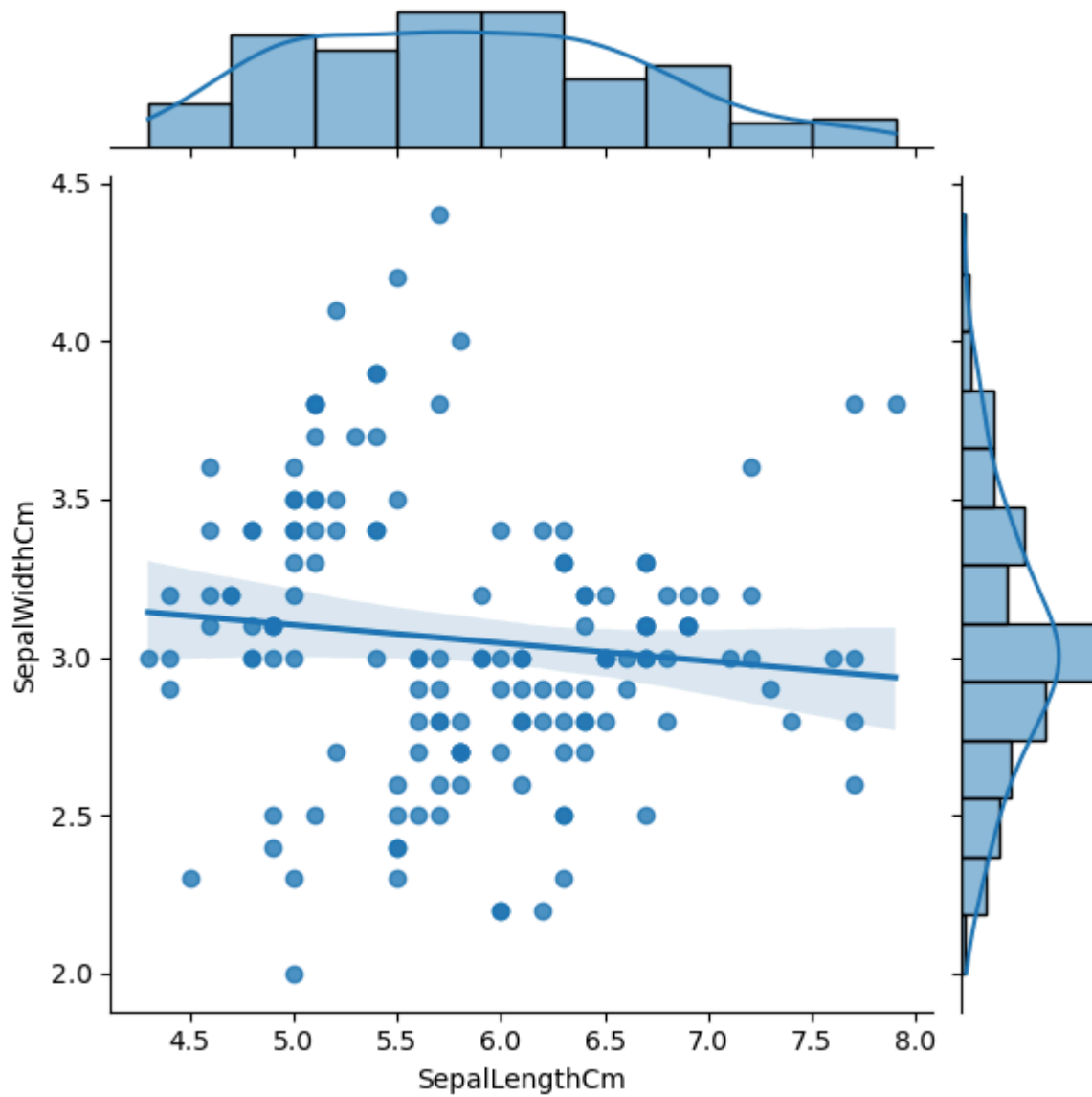
Out[15]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

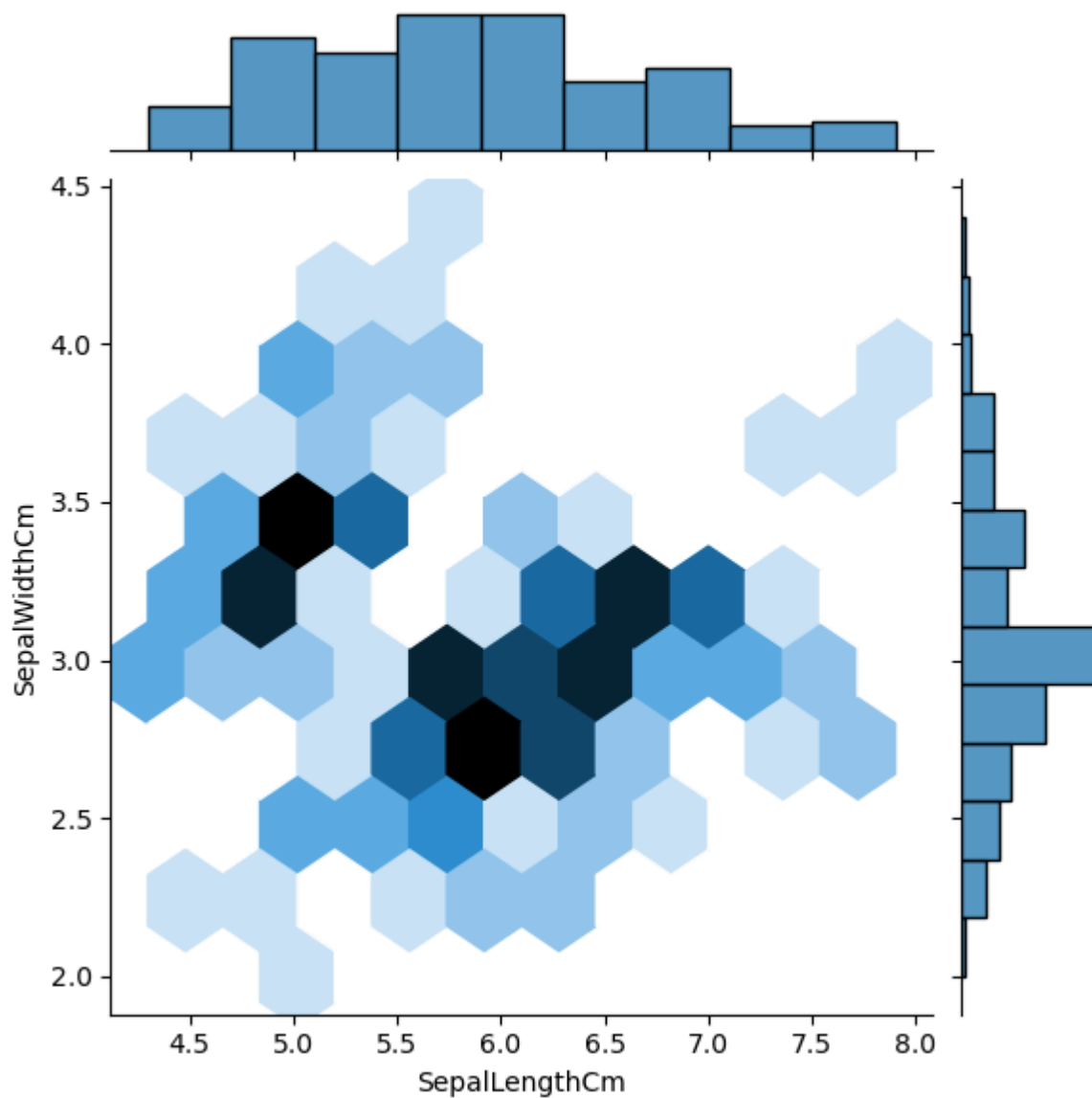
In [16]: `fig=sns.jointplot(x='SepalLengthCm',y='SepalWidthCm',data=iris)`



In [18]: `sns.jointplot(x="SepalLengthCm", y="SepalWidthCm", data=iris, kind="reg")`
`plt.show()`



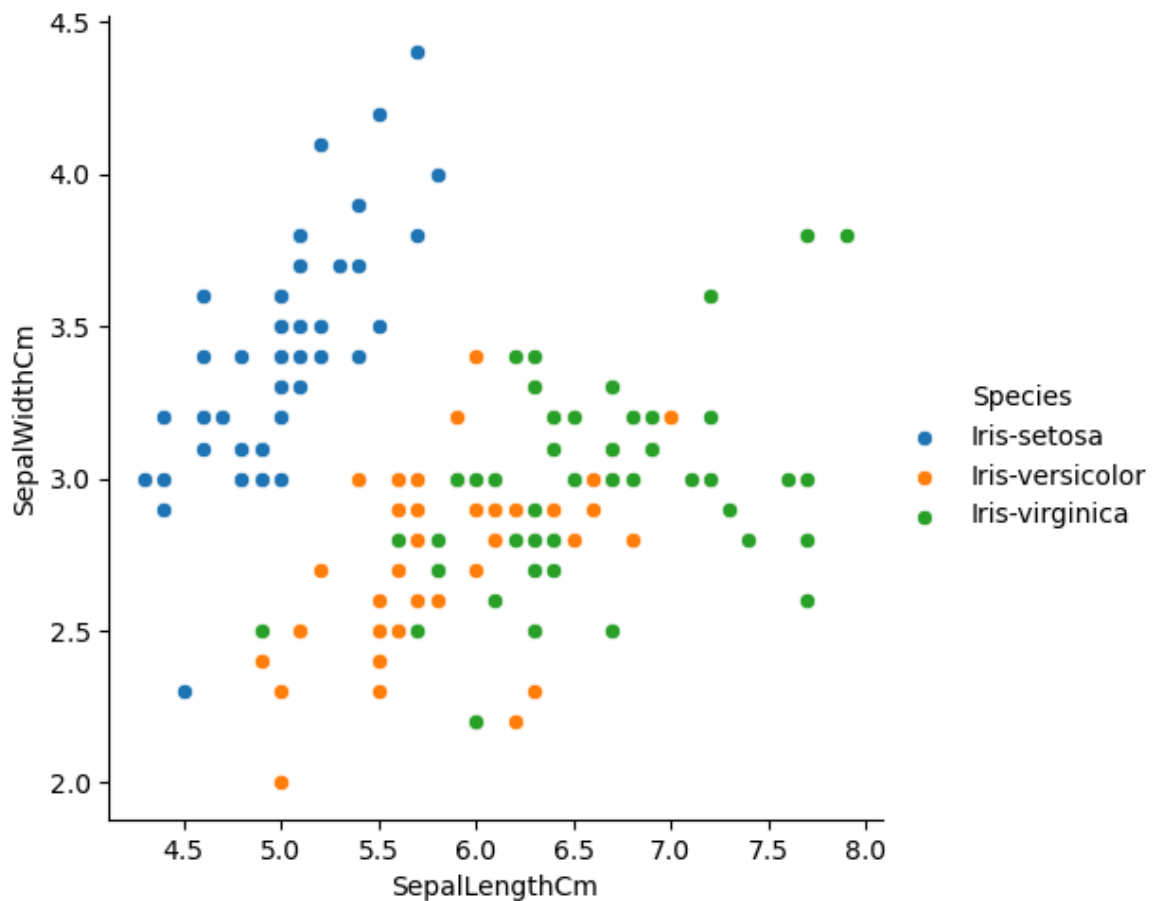
```
In [19]: fig=sns.jointplot(x='SepalLengthCm',y='SepalWidthCm',kind='hex',data=iris)
```



5. FacetGrid Plot

```
In [22]: import matplotlib.pyplot as plt
%matplotlib inline

g = sns.FacetGrid(iris, hue='Species', height=5)
g.map_dataframe(sns.scatterplot, x='SepalLengthCm', y='SepalWidthCm')
g.add_legend()
plt.show()
```



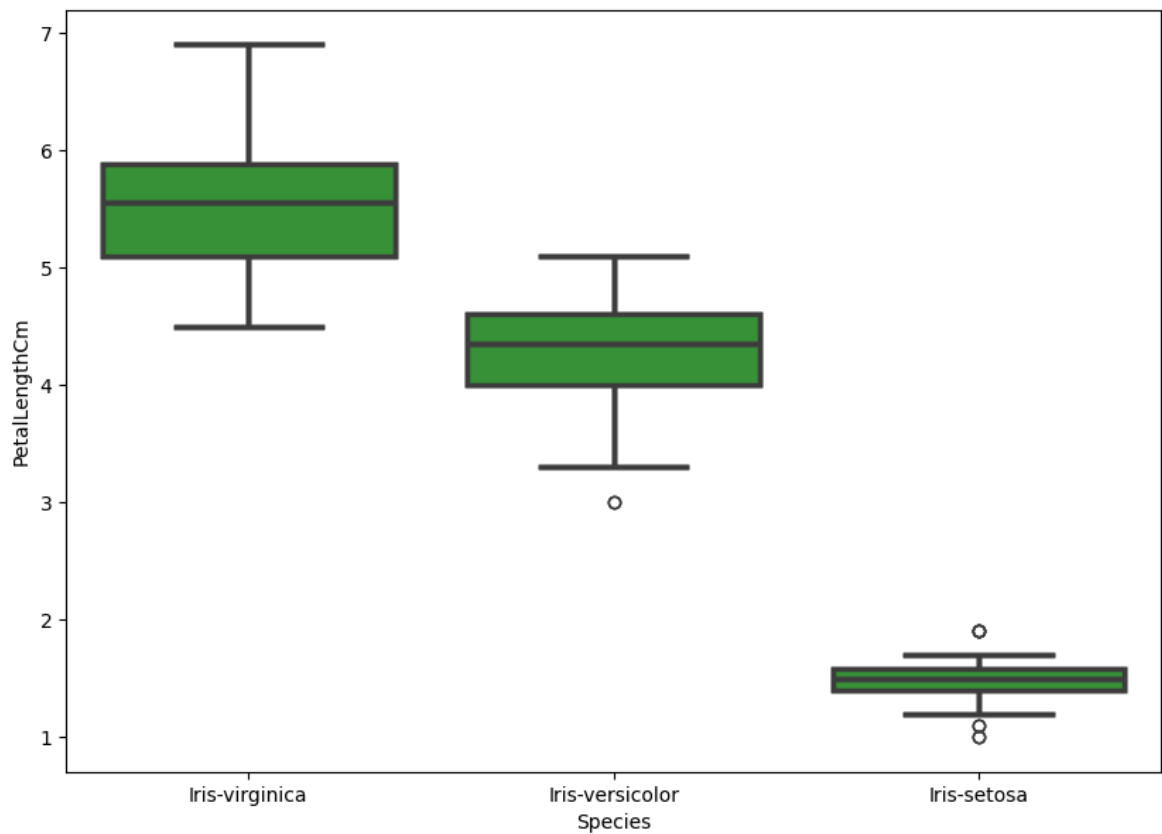
6. Boxplot or Whisker plot Box plot was first introduced in year 1969 by Mathematician John Tukey. Box plot give a statcal summary of the features being plotted. Top line represent the max value, top edge of box is third Quartile, middle edge represents the median, bottom edge represents the first quartile value. The bottom most line represent the minimum value of the feature. The height of the box is called as Interquartile range. The black dots on the plot represent the outlier values in the data.

```
In [23]: iris.head()
```

```
Out[23]:
```

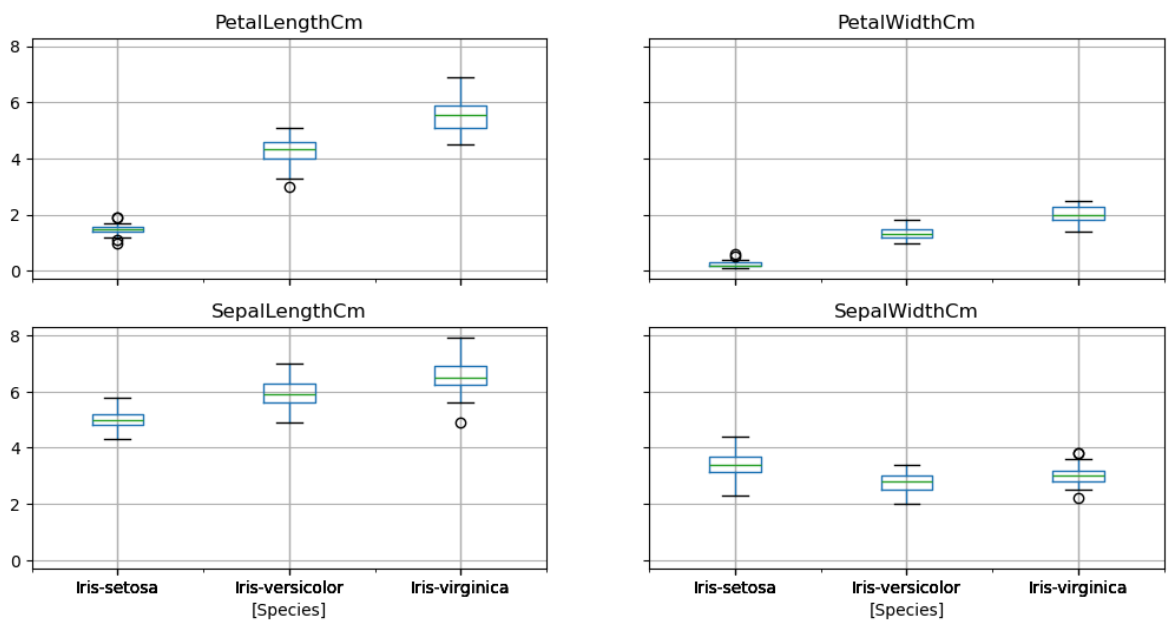
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

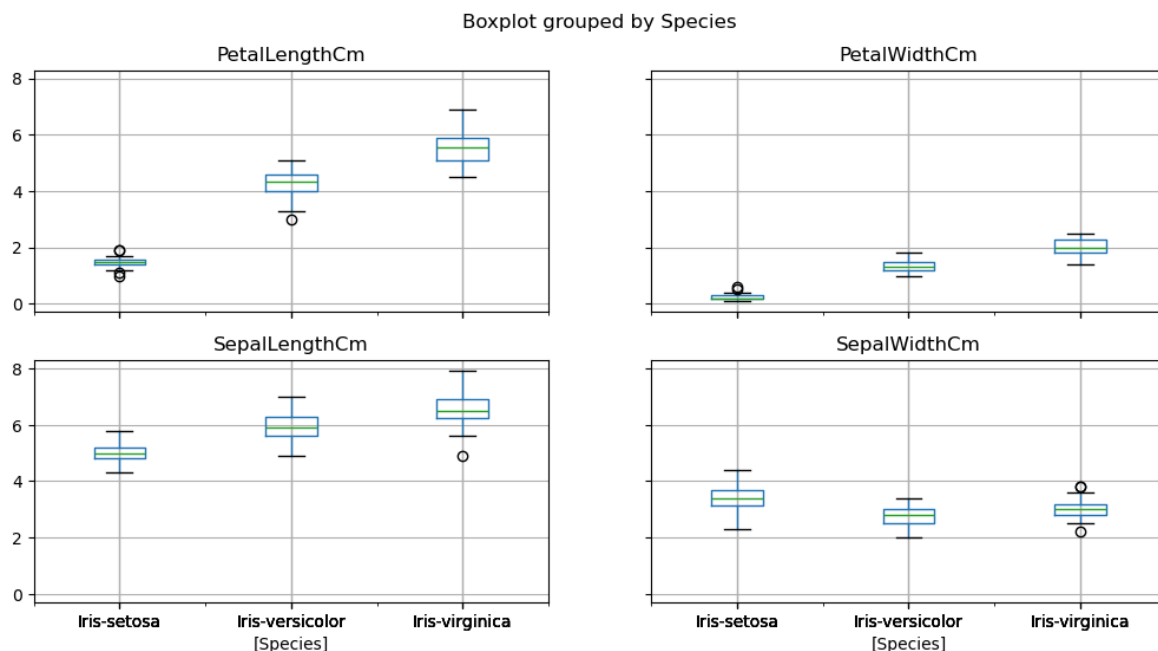
```
In [26]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.boxplot(x='Species',y='PetalLengthCm',data=iris,order=['Iris-virginica',
plt.show()
```



```
In [28]: #iris.drop("Id", axis=1).boxplot(by="Species", figsize=(12, 6))
iris.boxplot(by="Species", figsize=(12, 6))
plt.show()
```

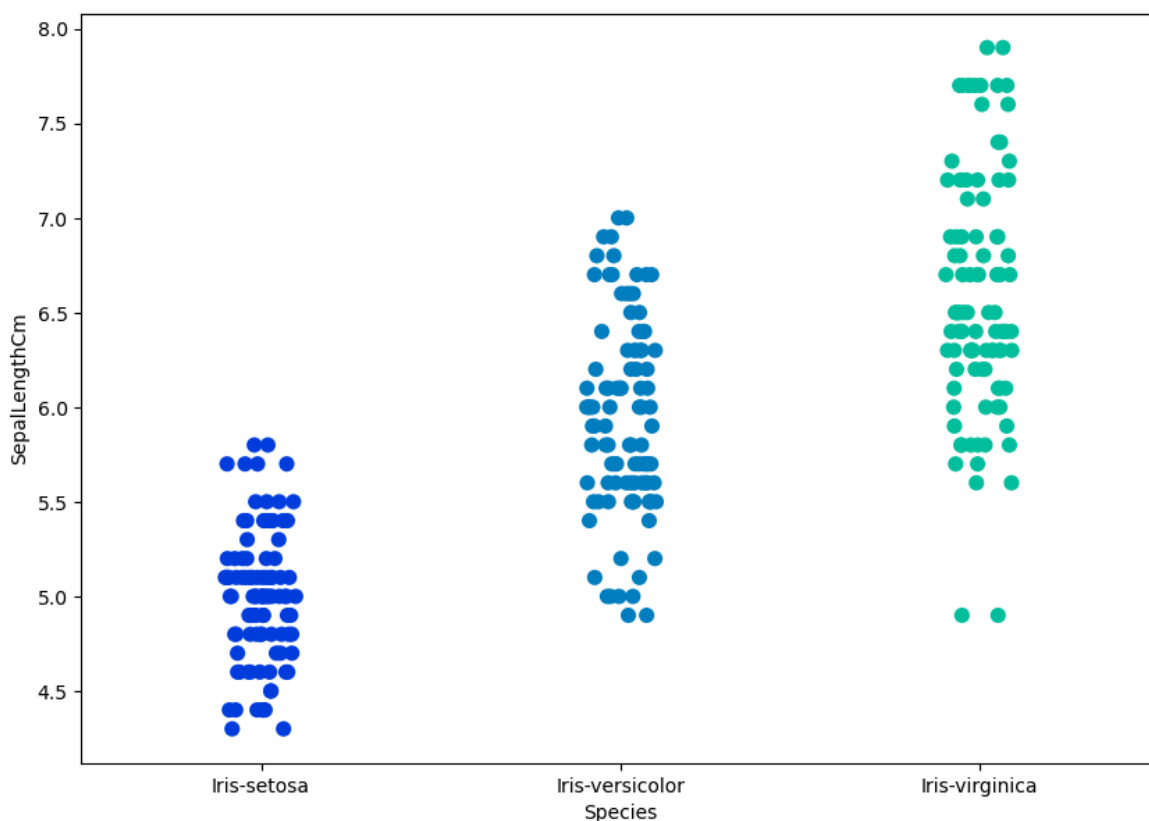
Boxplot grouped by Species





7. Strip plot

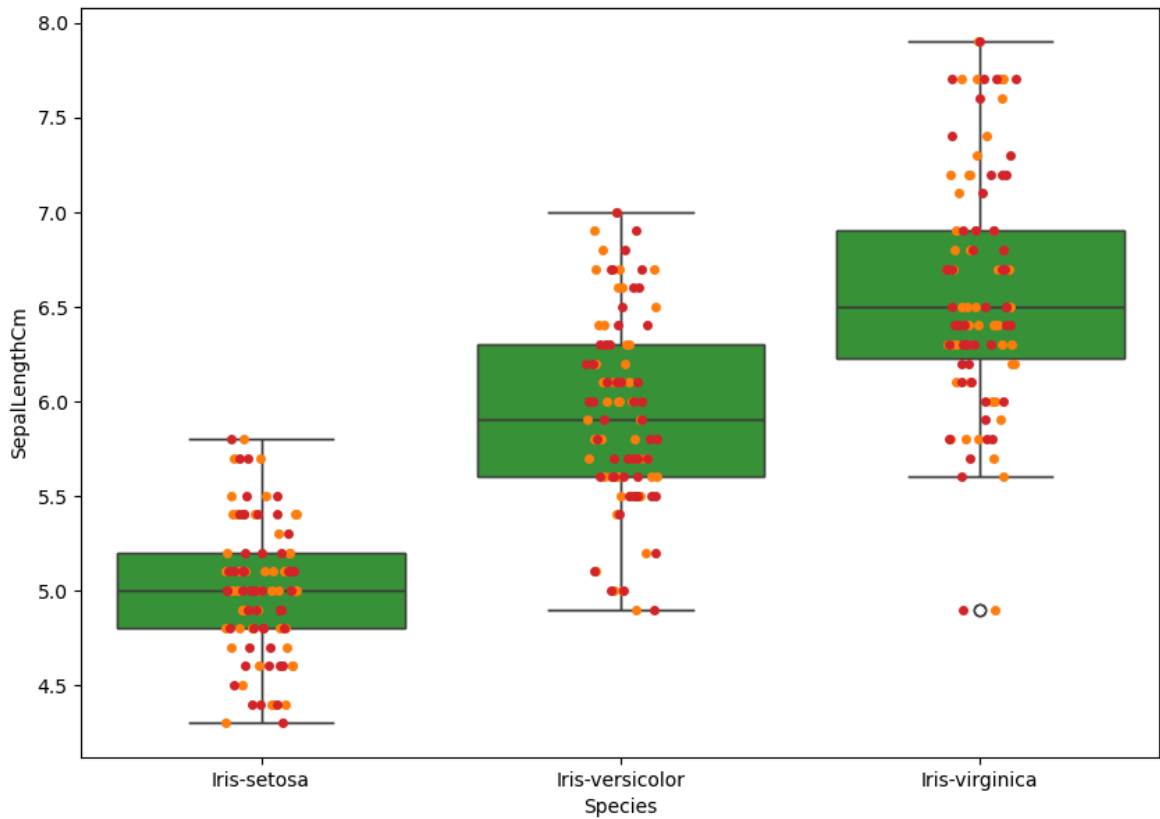
```
In [30]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.stripplot(x='Species',y='SepalLengthCm',data=iris,jitter=True,edgecolor=
plt.show())
```



8. Combining Box and Strip Plots

```
In [33]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.boxplot(x='Species',y='SepalLengthCm',data=iris)
```

```
fig=sns.stripplot(x='Species',y='SepalLengthCm',data=iris,jitter=True,edgecolor=
plt.show()
```



```
In [35]: ax = sns.boxplot(x="Species", y="PetalLengthCm", data=iris)
sns.stripplot(x="Species", y="PetalLengthCm", data=iris, jitter=True, edgecolor=

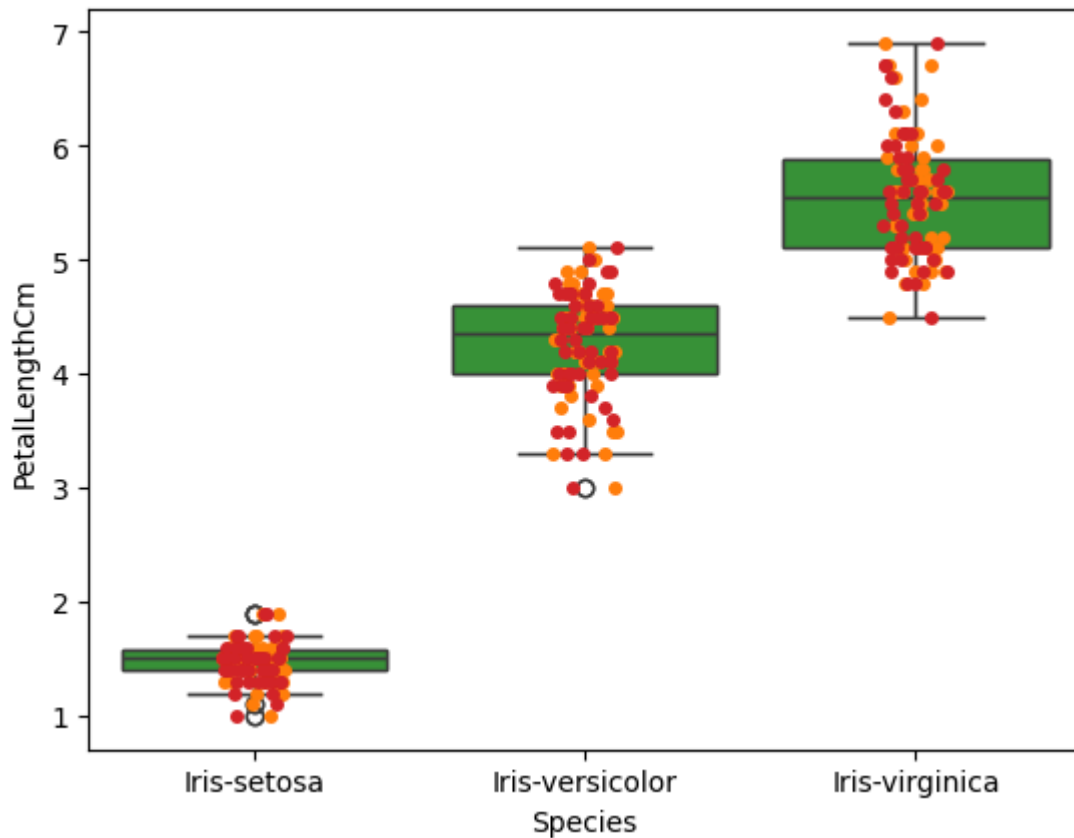
# Access box patches (each species = one box)
boxes = ax.patches

# Color each box
boxes[0].set_facecolor("green")
boxes[0].set_edgecolor("black")

boxes[1].set_facecolor("red")
boxes[1].set_edgecolor("black")

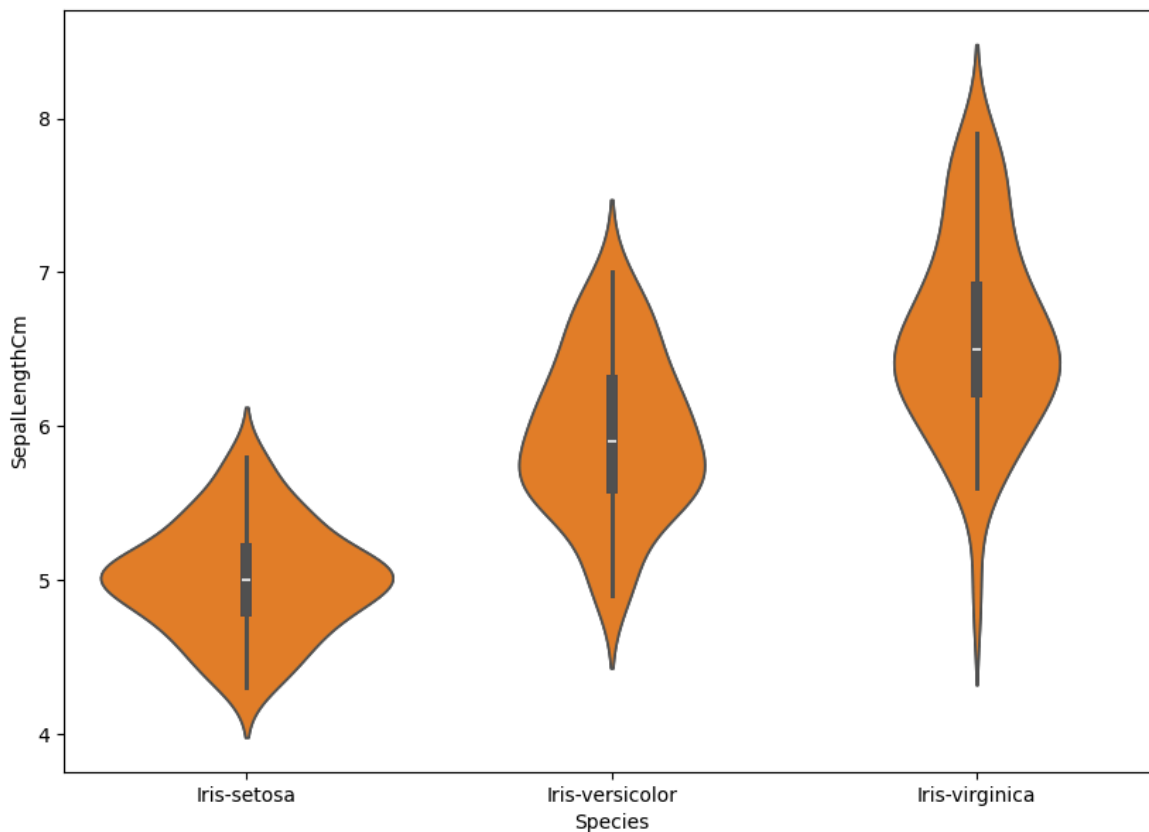
boxes[2].set_facecolor("yellow")
boxes[2].set_edgecolor("black")

plt.show()
```



9. Violin Plot It is used to visualize the distribution of data and its probability distribution. This chart is a combination of a Box Plot and a Density Plot that is rotated and placed on each side, to show the distribution shape of the data. The thick black bar in the centre represents the interquartile range, the thin black line extended from it represents the 95% confidence intervals, and the white dot is the median. Box Plots are limited in their display of the data, as their visual simplicity tends to hide significant details about how values in the data are distributed

```
In [37]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.violinplot(x='Species',y='SepalLengthCm',data=iris)
plt.show()
```



```
In [49]: import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(15, 10))

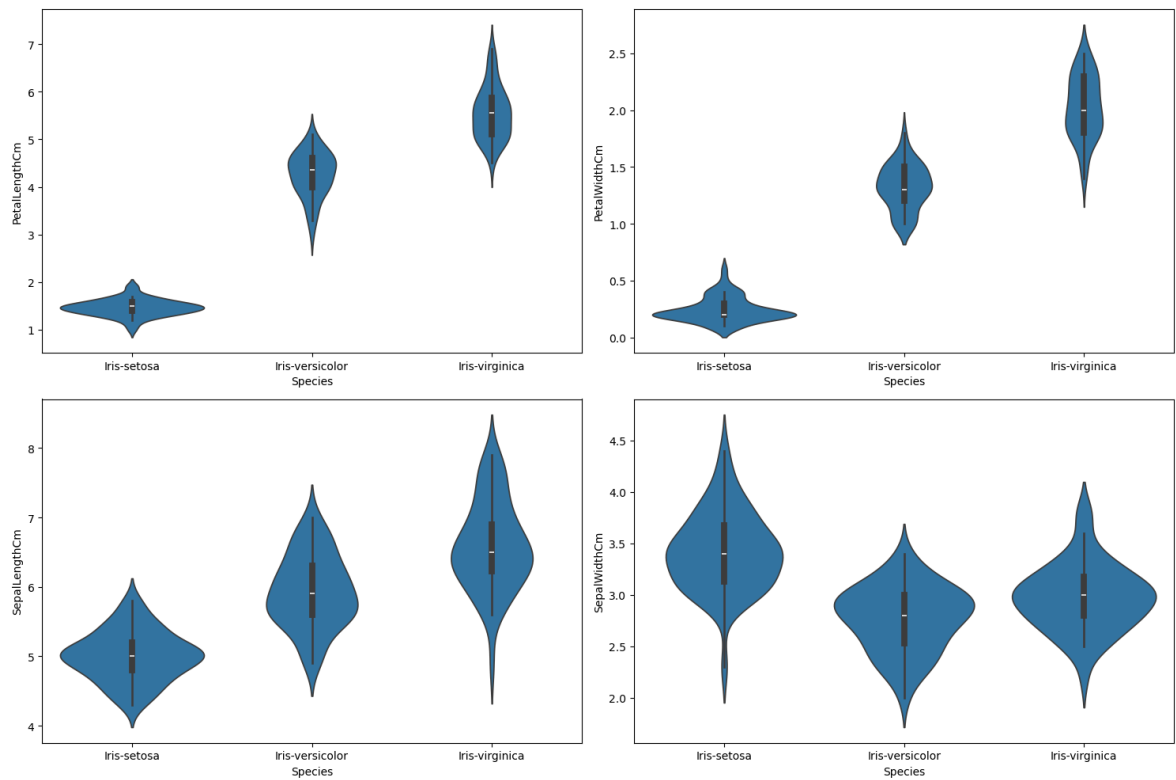
plt.subplot(2, 2, 1)
sns.violinplot(data=iris, x='Species', y='PetalLengthCm')

plt.subplot(2, 2, 2)
sns.violinplot(data=iris, x='Species', y='PetalWidthCm')

plt.subplot(2, 2, 3)
sns.violinplot(data=iris, x='Species', y='SepalLengthCm')

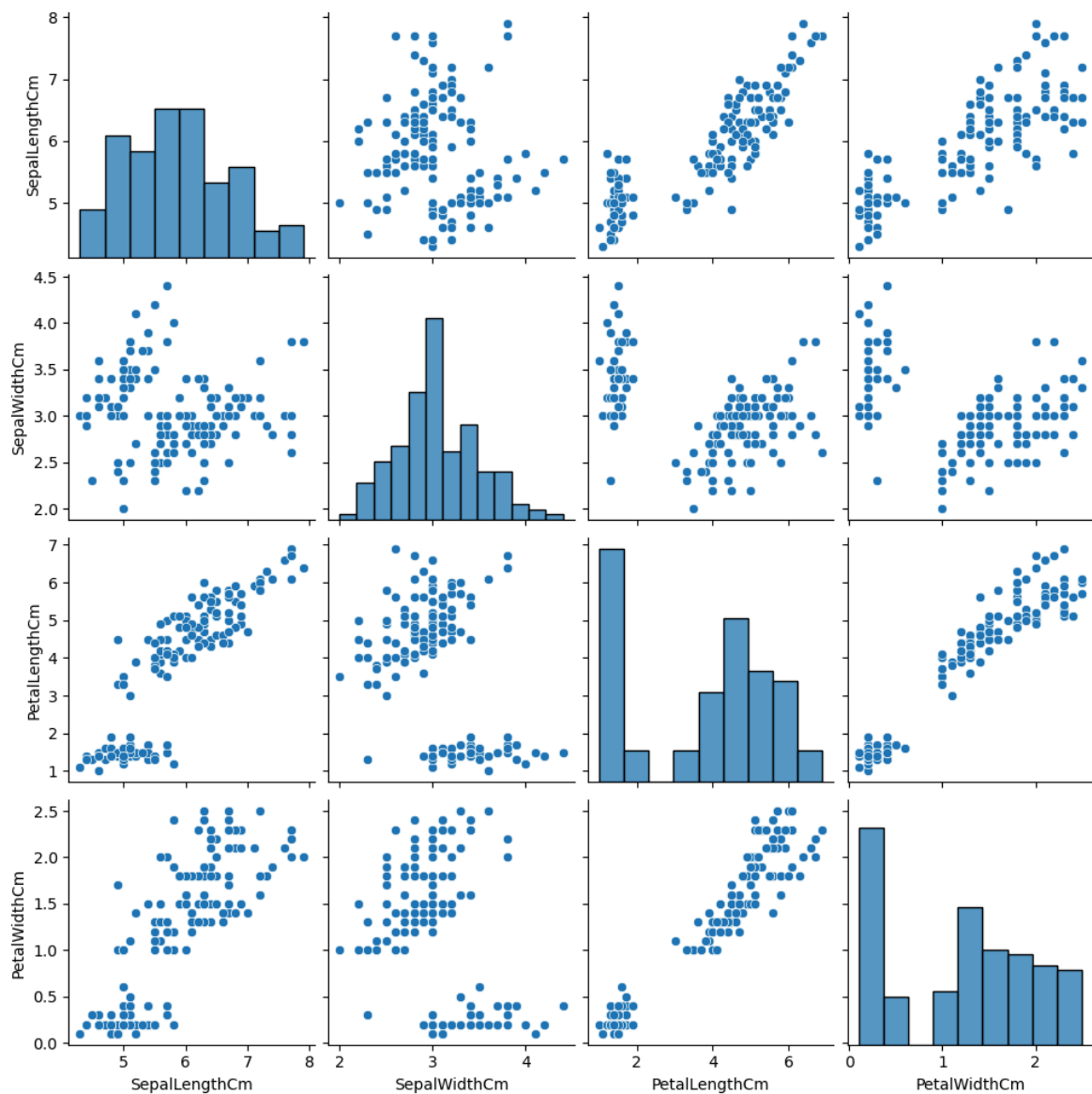
plt.subplot(2, 2, 4)
sns.violinplot(data=iris, x='Species', y='SepalWidthCm')

plt.tight_layout()
plt.show()
```

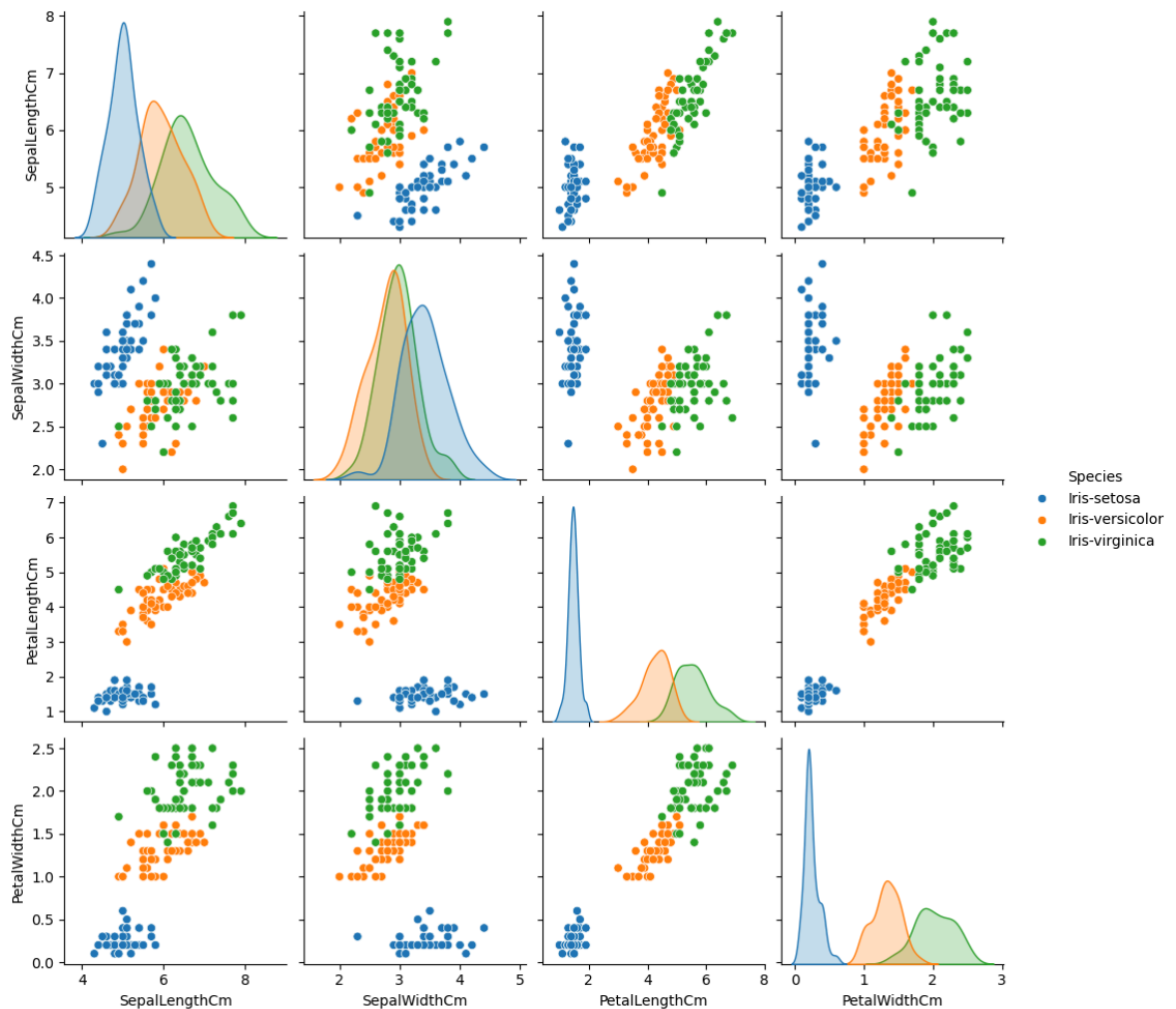


10. Pair Plot: A "pairs plot" is also known as a scatterplot, in which one variable in the same data row is matched with another variable's value, like this: Pairs plots are just elaborations on this, showing all variables paired with all the other variables.

```
In [50]: sns.pairplot(data=iris)
plt.show()
```



```
In [51]: sns.pairplot(data=iris, hue="Species")  
plt.show()
```



11. Heat map Heat map is used to find out the correlation between different features in the dataset. High positive or negative value shows that the features have high correlation. This helps us to select the parameters for machine learning.

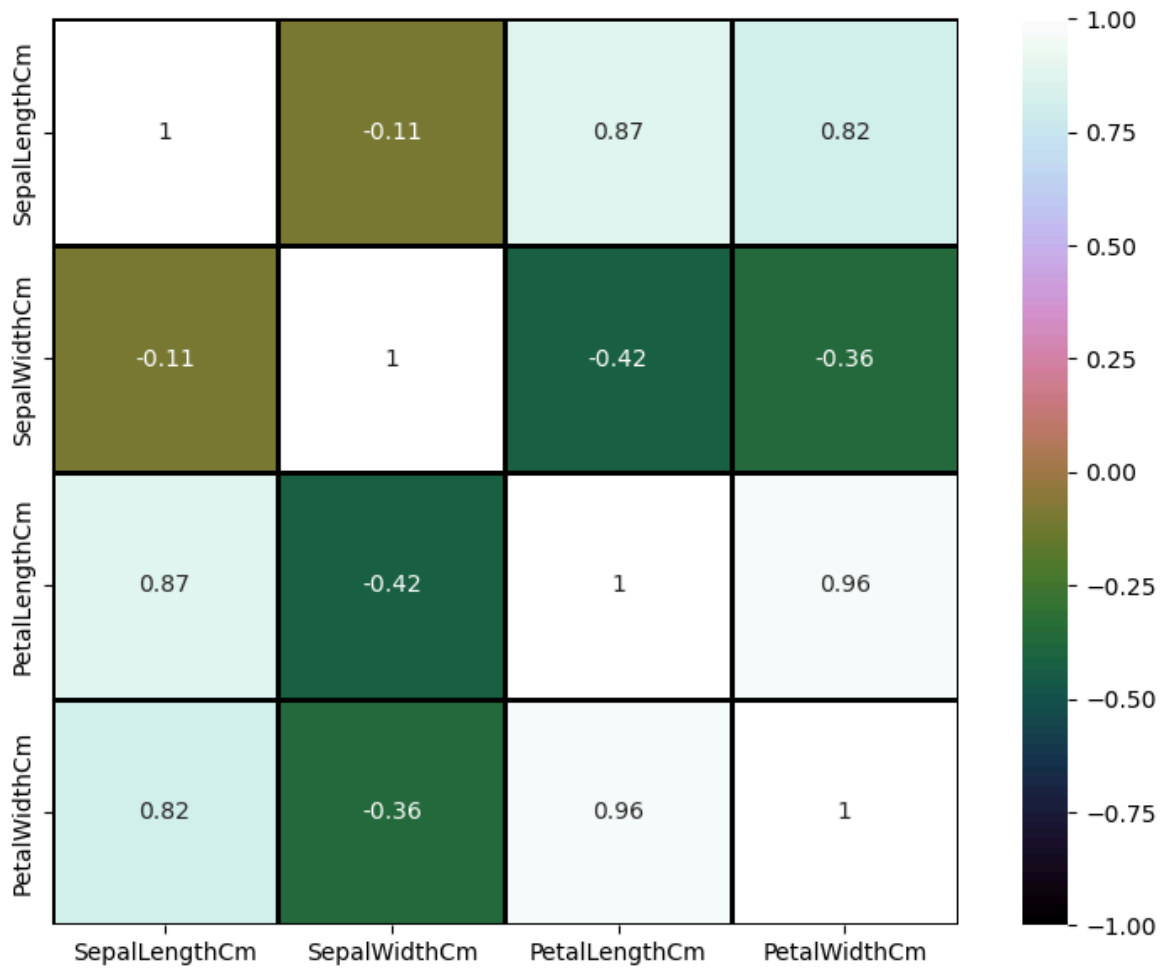
```
In [53]: # Select only numeric columns for correlation
numeric_iris = iris.select_dtypes(include='number')

plt.figure(figsize=(10, 7))

sns.heatmap(
    numeric_iris.corr(),
    annot=True,
    cmap='cubehelix',
    linewidths=1,
    linecolor='k',
    square=True,
    vmin=-1,
    vmax=1,
    cbar_kws={"orientation": "vertical"}
)

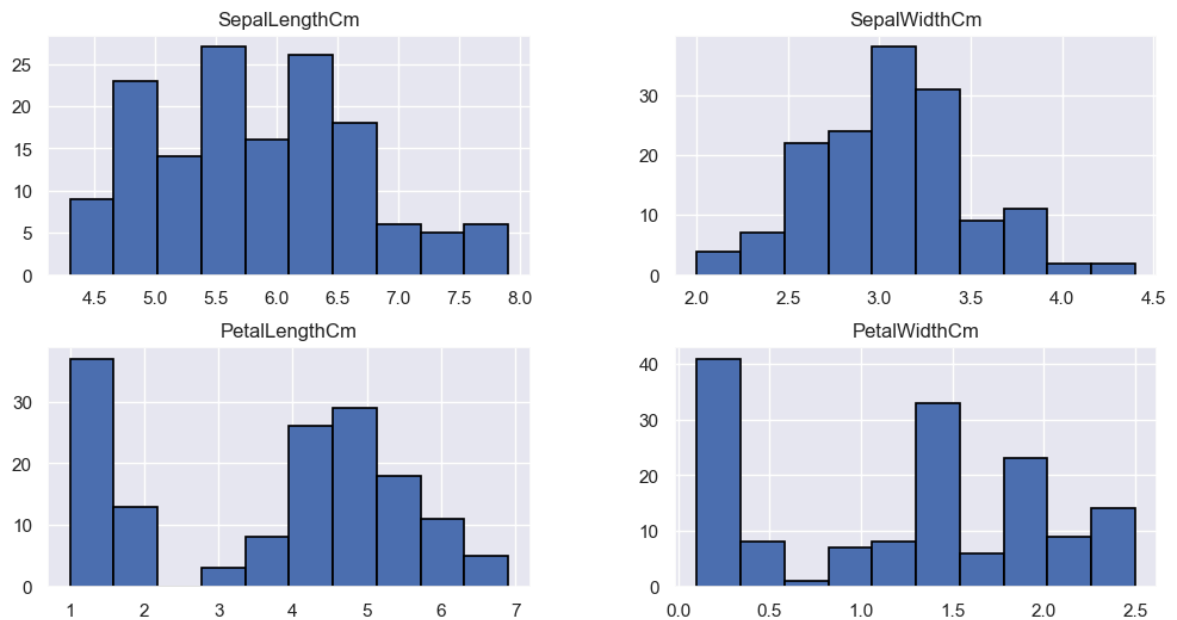
plt.show()
```

<Figure size 1000x700 with 0 Axes>



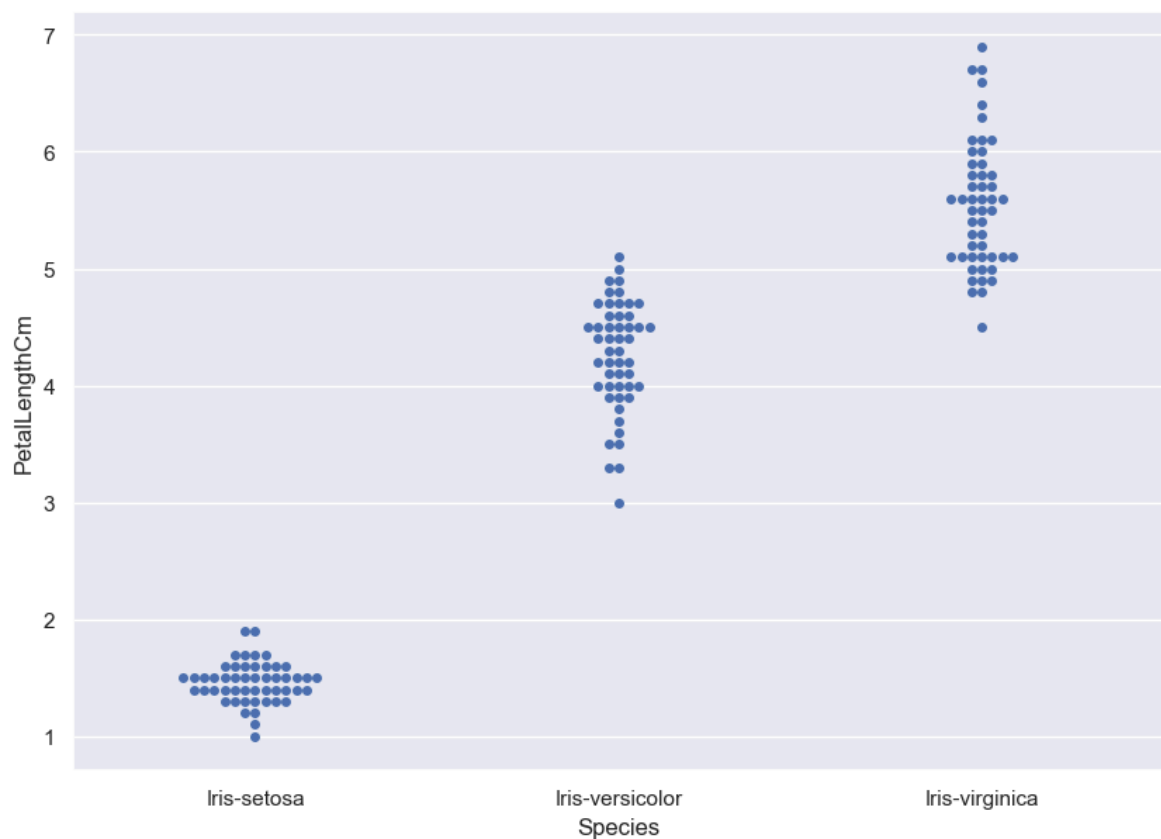
12. Distribution plot: The distribution plot is suitable for comparing range and distribution for groups of numerical data. Data is plotted as value points along an axis. You can choose to display only the value points to see the distribution of values, a bounding box to see the range of values, or a combination of both as shown here. The distribution plot is not relevant for detailed analysis of the data as it deals with a summary of the data distribution.

```
In [59]: iris.hist(edgecolor='black', linewidth=1.2)
fig = plt.gcf()
fig.set_size_inches(12, 6)
plt.show()
```

13. Swarm plot It looks a bit like a friendly swarm of bees buzzing about their hive. More importantly, each data point is clearly visible and no data are obscured by overplotting. A beeswarm plot improves upon the random jittering approach to move data points the minimum distance away from one another to avoid overlays. The result is a plot where you can see each distinct data point, like shown in below plot

```
In [58]: sns.set(style="darkgrid")
plt.figure(figsize=(10, 7))
sns.swarmplot(x="Species", y="PetalLengthCm", data=iris)
plt.show()
```



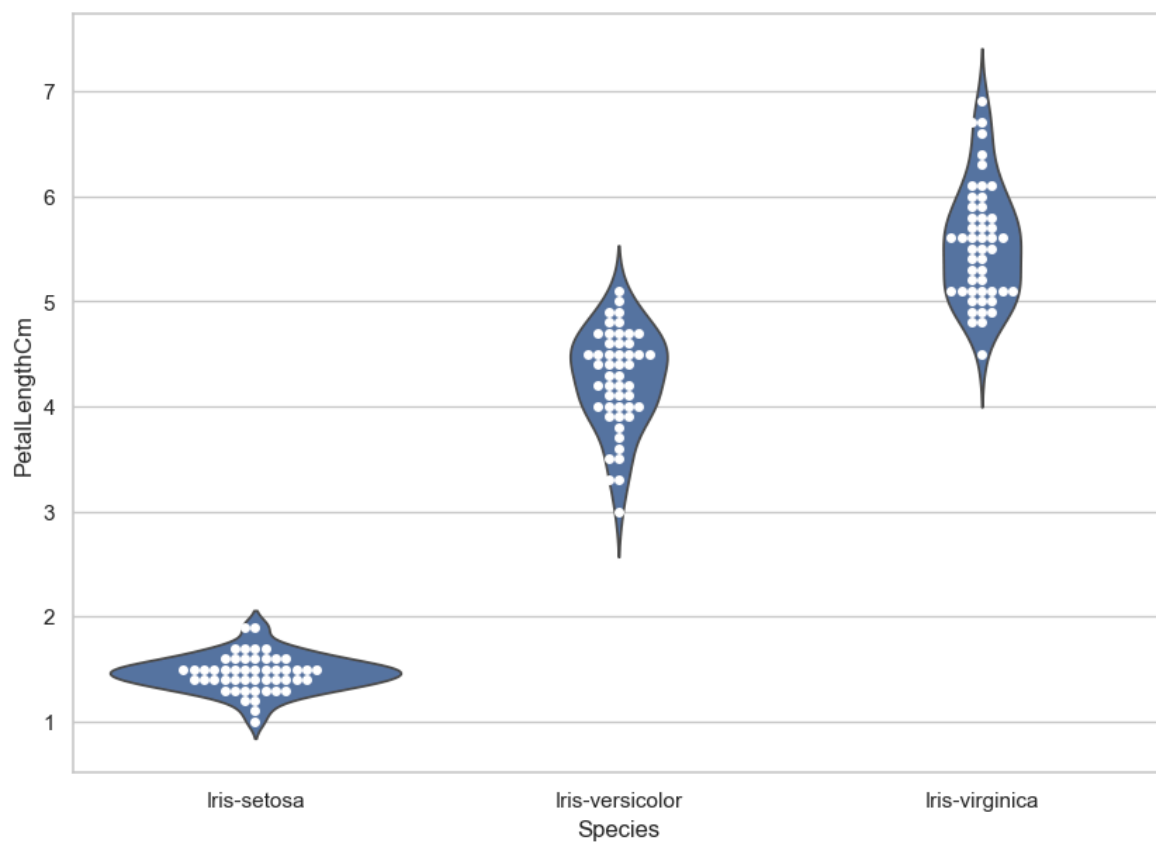
```
In [60]: sns.set(style="whitegrid")
```

```
plt.figure(figsize=(10, 7))

# Violin plot
sns.violinplot(
    x="Species",
    y="PetalLengthCm",
    data=iris,
    inner=None
)

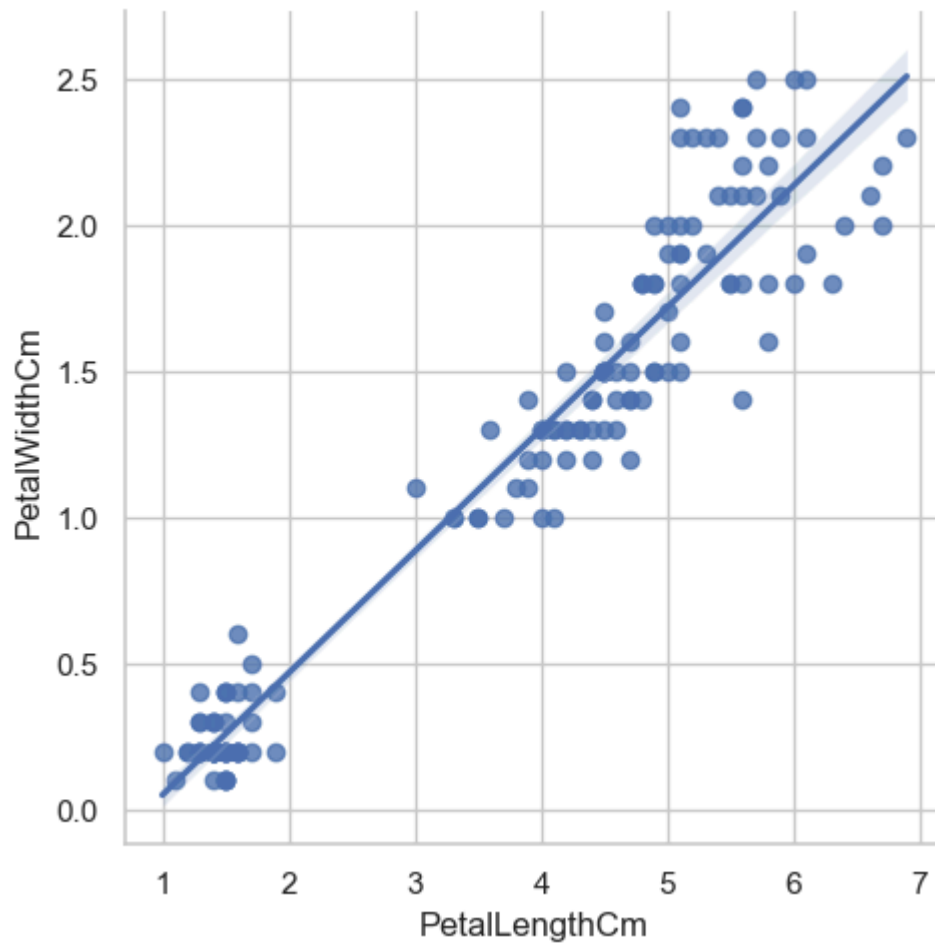
# Swarm plot on top
sns.swarmplot(
    x="Species",
    y="PetalLengthCm",
    data=iris,
    color="white",
    edgecolor="black"
)

plt.show()
```



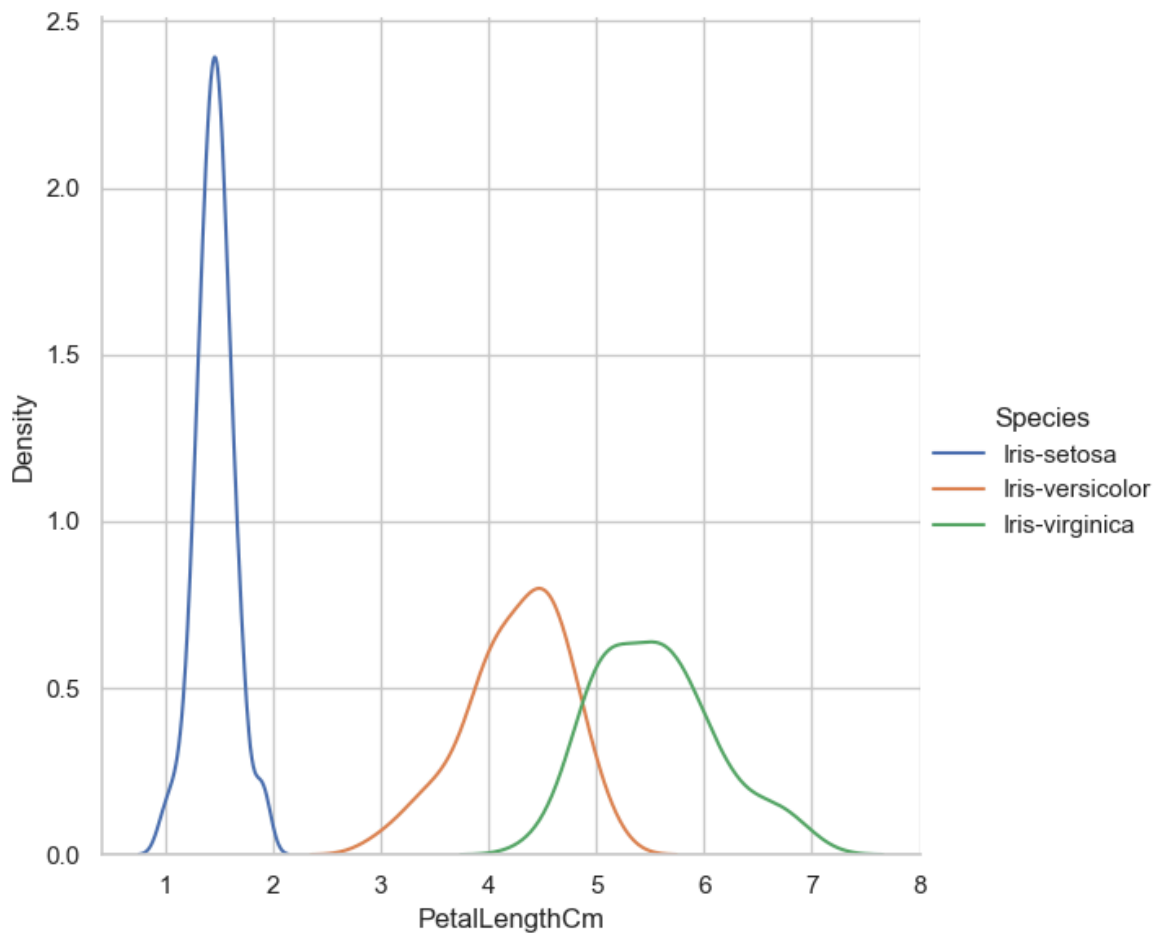
17. LM Plot

```
In [61]: sns.lmplot(x="PetalLengthCm", y="PetalWidthCm", data=iris)
plt.show()
```



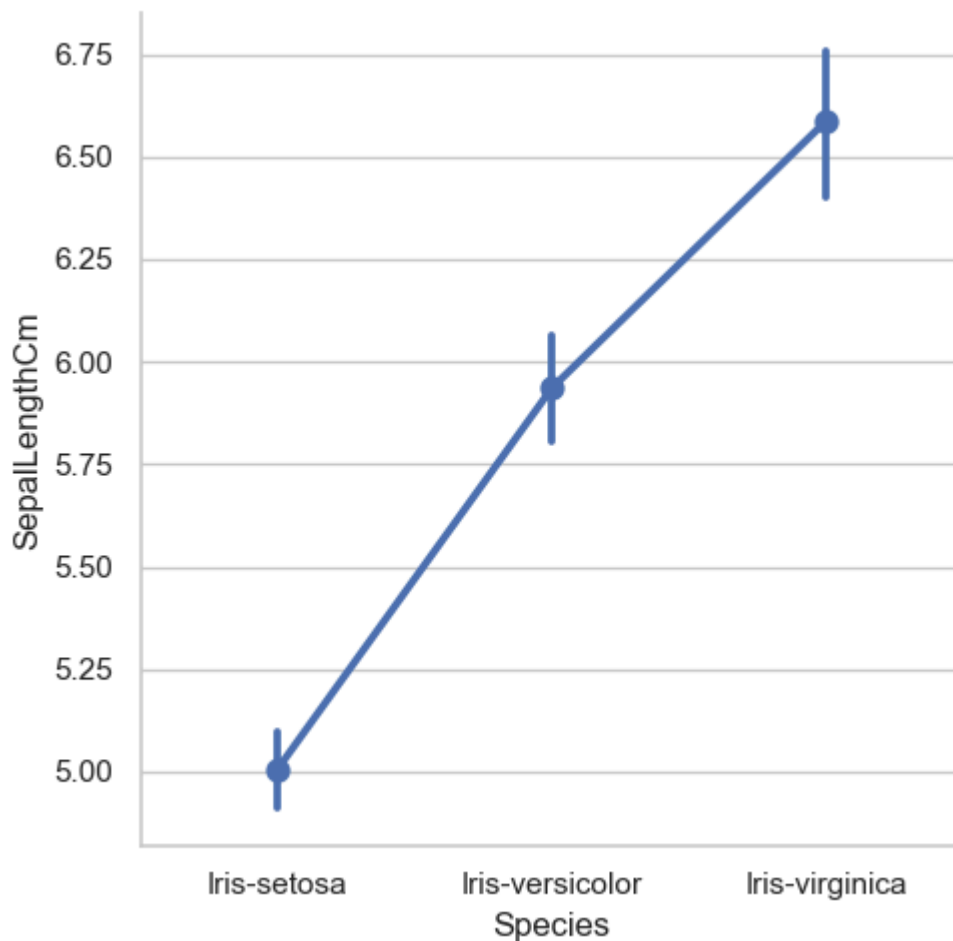
18. FacetGrid

```
In [62]: g = sns.FacetGrid(iris, hue="Species", height=6)
g.map_dataframe(sns.kdeplot, x="PetalLengthCm", fill=False)
g.add_legend()
plt.show()
```



22. Factor Plot

```
In [63]: sns.catplot(x="Species", y="SepalLengthCm", data=iris, kind="point")  
plt.show()
```



23. Boxen Plot

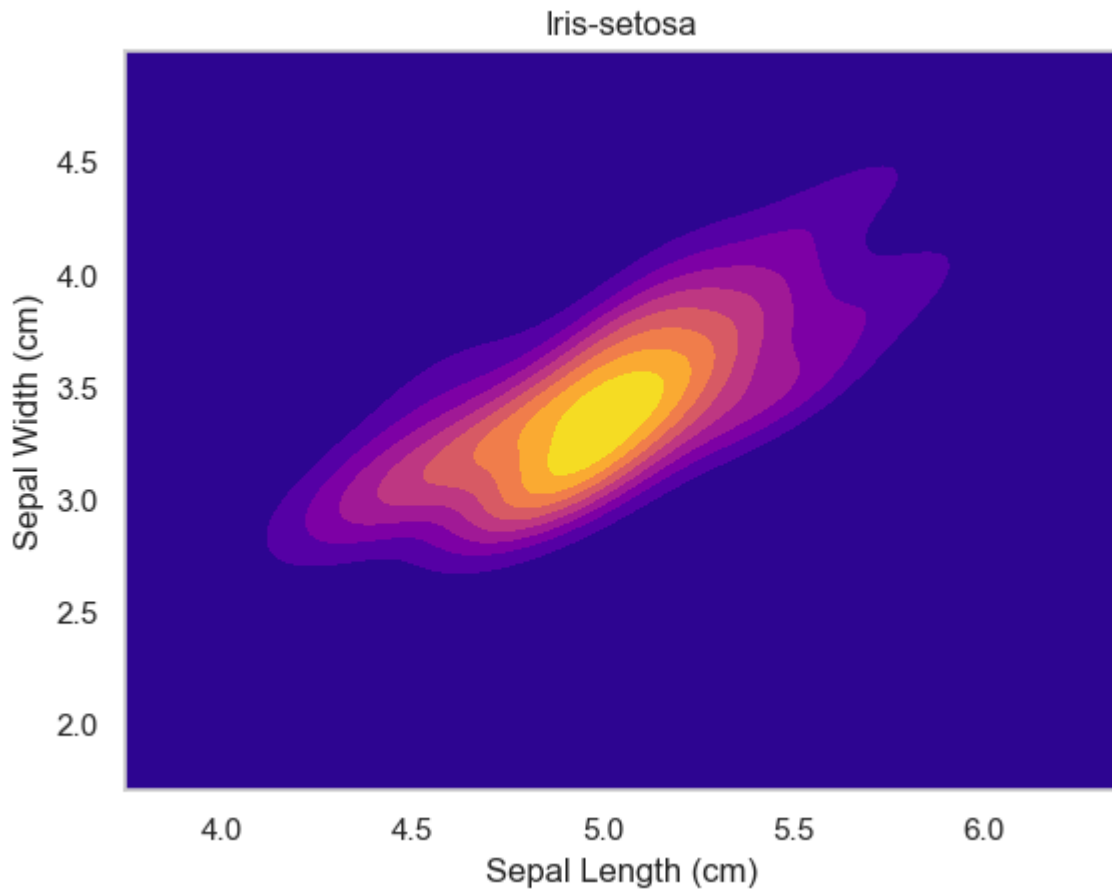
```
In [ ]: plt.figure(figsize=(10, 7))
sns.boxenplot(x='Species', y='SepalLengthCm', data=iris)
plt.show()
```

28.KDE Plot

```
In [65]: # Filter only setosa
sub = iris[iris['Species'] == 'Iris-setosa']

# KDE plot
sns.kdeplot(
    data=sub,
    x='SepalLengthCm',
    y='SepalWidthCm',
    cmap="plasma",
    fill=True,
    thresh=0
)

plt.title('Iris-setosa')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
```



30. Dashboard

```
In [66]: sns.set_style('darkgrid')

f, axes = plt.subplots(2, 2, figsize=(15, 15))

# Boxplot
sns.boxplot(
    x="Species",
    y="PetalLengthCm",
    data=iris,
    ax=axes[0, 0]
)

# Violin plot
sns.violinplot(
    x='Species',
    y='PetalLengthCm',
    data=iris,
    ax=axes[0, 1]
)

# Strip plot
sns.stripplot(
    x='Species',
    y='SepalLengthCm',
    data=iris,
    jitter=True,
    edgecolor='gray',
    size=8,
```

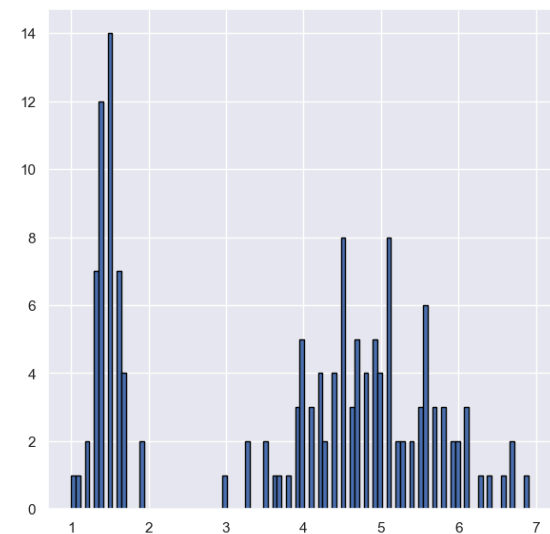
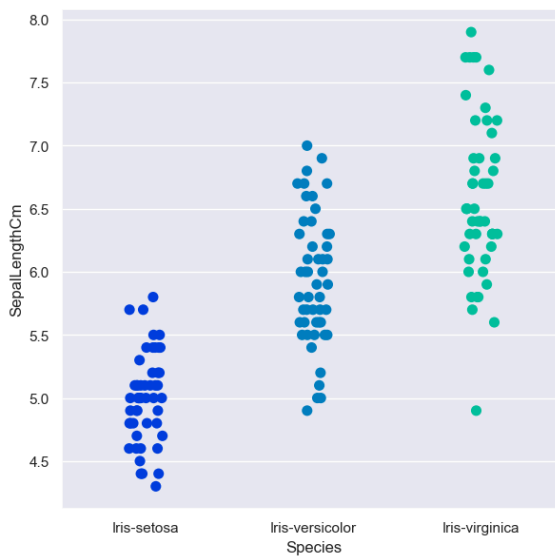
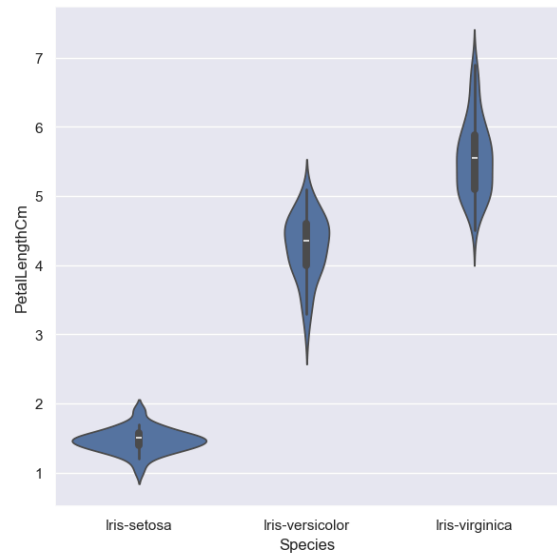
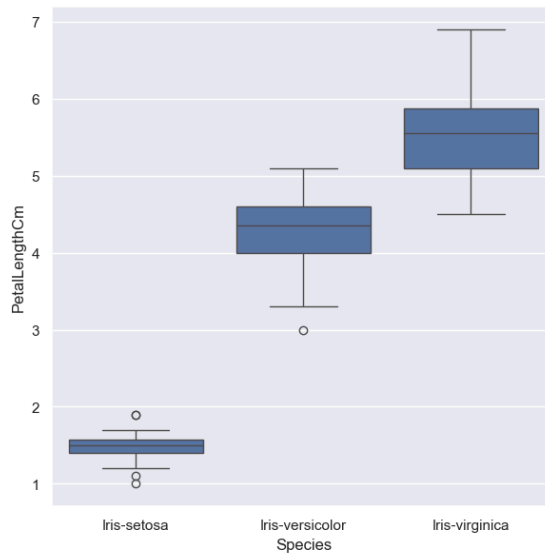
```

palette='winter',
orient='v',
ax=axes[1, 0]
)

# Histogram
axes[1, 1].hist(iris['PetalLengthCm'], bins=100, edgecolor='black')

plt.show()

```



In the dashboard we have shown how to create multiple plots to form a dashboard using Python. In this plot we have demonstrated how to plot Seaborn and Matplotlib plots on the same Dashboard.

31. Stacked Histogram

```

In [68]: iris['Species'] = iris['Species'].astype('category')
iris.head()

```

Out[68]:

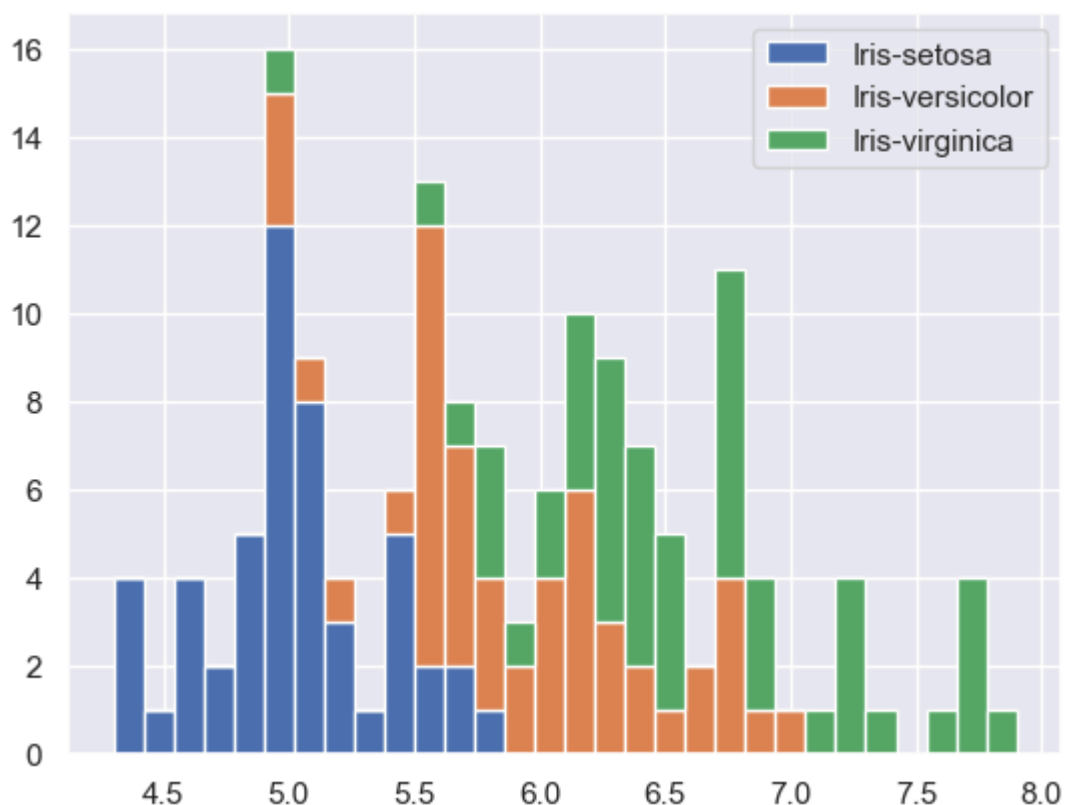
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [69]: # Ensure Species is categorical
iris['Species'] = iris['Species'].astype('category')

list1 = []
mylabels = []

for gen in iris['Species'].cat.categories:
    list1.append(iris[iris['Species'] == gen]['SepalLengthCm'])
    mylabels.append(gen)

plt.hist(list1, bins=30, stacked=True, rwidth=1, label=mylabels)
plt.legend()
plt.show()
```



With Stacked Histogram we can see the distribution of Sepal Length of Different Species together. This shows us the range of Sepal Length for the three different Species of Iris Flower.

32.Area Plot

Area Plot gives us a visual representation of Various dimensions of Iris flower and their range in dataset.

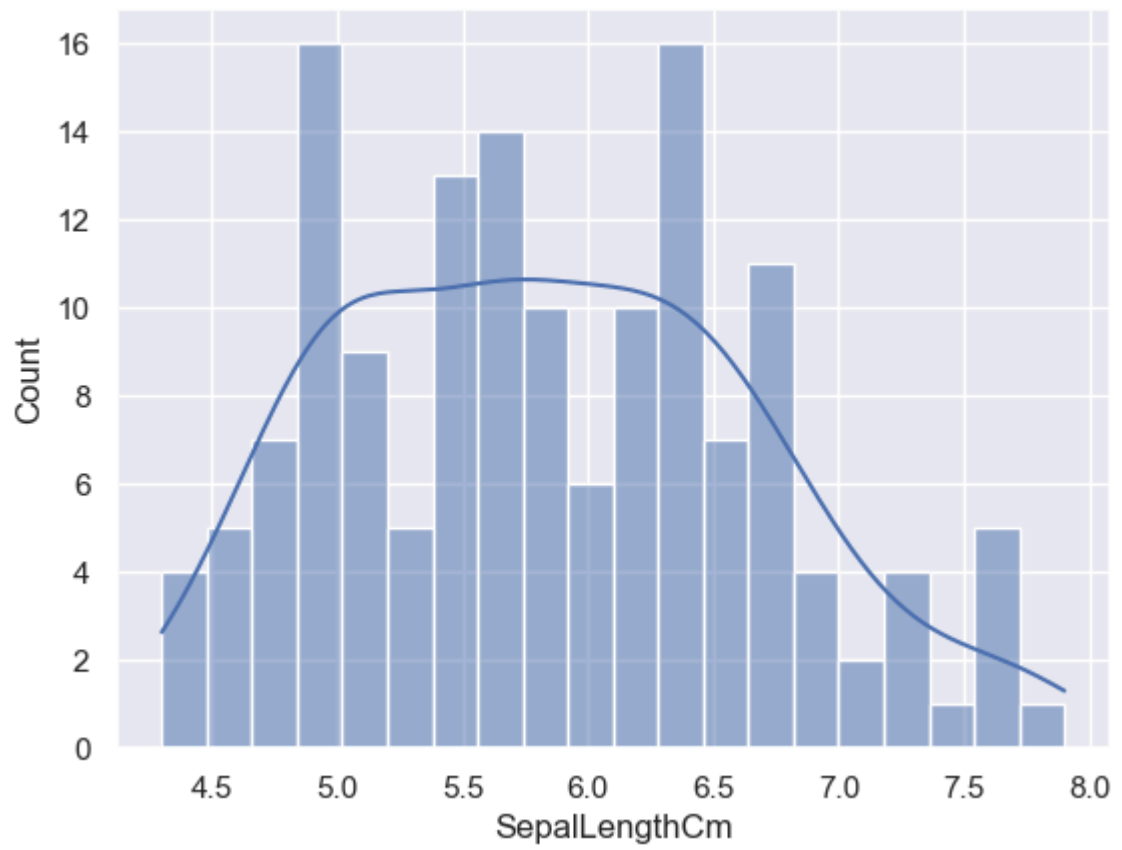
```
In [70]: iris.plot.area(  
    y=['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm'],  
    alpha=0.4,  
    figsize=(12, 6)  
)  
  
plt.title("Area Plot of Iris Measurements")  
plt.xlabel("Index")  
plt.ylabel("Centimeters")  
plt.show()
```



33.Distplot

It helps us to look at the distribution of a single variable.Kde shows the density of the distribution

```
In [71]: sns.histplot(iris['SepalLengthCm'], bins=20, kde=True)  
plt.show()
```



In []: